

Generative AI for Diagrams as Code and Code as Diagrams

Mircea Trifan, Bogdan Ionescu and Dan Ionescu

School of Electrical Engineering and Computer Science, University of Ottawa,
Ottawa, Ontario, Canada

Email: {mircea, bogdan, dan}@ncct.uottawa.ca

Abstract—Modern software projects often involve extensive codebases that are difficult to navigate, document, and keep consistent across multiple teams and repositories. Traditional documentation often becomes outdated, leading to inconsistencies and unclear architectural insights. To address this, Generative AI and Large Language Models (LLMs) such as ChatGPT and Claude automate the bidirectional transformation between code and diagrams, seamlessly integrating into software workflows. The proposed framework uses PlantUML and TikZ, to improve debugging, documentation, and versioning. Neo4j is used for natural language retrieval, and advanced queries using Cypher as well as similarity searches. This hybrid approach mitigates LLM-related challenges like hallucinations by combining database integration with prompt engineering. By consolidating code, diagrams, and metadata into a unified resource, the framework aims to improve maintainability, collaboration, and transparency, as demonstrated in initial qualitative use cases.

Index Terms—Diagrams as Code, Code as Diagrams, LLMs, Claude, ChatGPT, eXplainable AI (XAI), C4, Neo4j, PlantUML, TikZ.

I. INTRODUCTION

Visualization plays a crucial role in the development and understanding of machine learning (ML) models. It not only aids in interpreting complex model behaviors but also helps in diagnosing problems and communicating findings effectively. As machine learning applications become more pervasive across various domains, the ability to clearly represent and explain these models becomes increasingly important.

Developed by Simon Brown, the C4 model [1] offers a structured method to visualize software architectures, emphasizing clarity through distinct layers of abstraction, to wit: Context, Containers, Components, and Code.

Our study expands the application of the C4 model to the domain of machine learning with application to real time and embedded software design. This paper introduces a novel framework that employs LLMs to generate executable and interpretable diagrams directly from code. This initiative is designed to enhance the accessibility of Python code by providing a visual representation of its models' architecture and data flows, thereby demystifying the underlying processes and aiding in both educational and debugging efforts. By translating intricate code structures into comprehensible visual formats, we address a critical gap in the interpretability of machine learning systems, thus broadening the utility and appeal of solutions across diverse applications. The advantages include streamlined versioning through GitHub integration and

reduced prompting requirements for diagram generation using LLMs. Diagrams as Code and Code as Diagrams share the goal of improving software visualization but approach the problem from opposite directions. The Diagrams as Code paradigm treats diagrams as maintainable, version-controlled artifacts, generated from structured text using tools like PlantUML and TikZ, ensuring they evolve alongside the software and integrate seamlessly into development pipelines. This approach enhances automation, consistency, and traceability, reducing the risk of outdated documentation. In contrast, Code as Diagrams transforms source code into graphical representations, aiding reverse engineering, debugging, and education by generating class diagrams, flowcharts, and dependency graphs dynamically. Rather than being mutually exclusive, these methods are complementary—Diagrams as Code provides forward compatibility by embedding architectural intent in the codebase, while Code as Diagrams enables backward analysis, reflecting real-time structural changes. By integrating both approaches, the framework ensures a bidirectional link between textual and visual representations, keeping documentation accurate, interpretable, and up to date throughout the software lifecycle. The paper is structured as follows: Section 2 reviews relevant literature. Section 3 details the architecture. Section 4 discusses the implementation of the "Diagrams as Code" approach, while Section 5 describes the reverse: "Code as Diagrams". Section 6 showcases examples of ChatGPT and Claude prompts for automating the interaction with the system. Section 7 describes the evaluation. The paper concludes in Section 8 with a summary of findings and future research directions.

II. RELATED WORKS

This section critically examines prior work in retrieval-augmented generation (RAG), diagram-based documentation, AI-assisted interpretability, and compositional software visualization. While many existing tools support the visualization of code artifacts or the maintenance of diagrams within software projects, none unify these capabilities with real-time LLM interaction, bi-directional transformations, and semantic querying through graph-based models such as Neo4j. This work addresses that gap.

Zimmermann's study [2] builds on existing advancements in chatbot and LLM development, emphasizing the integration of RAG techniques. By combining real-time retrieval of con-

textual information with large language models, this approach enhances chatbot performance and mitigates limitations related to outdated or missing knowledge in static LLMs.

The "Technology Radar" study [3] highlights the adoption of RAG techniques and explores diagrams as code, focusing on platforms like IcePanel and Structurizr that streamline collaborative architectural modeling through diagram versioning, object reuse, and flow visualization, improving alignment between technical and business stakeholders compared to traditional tools.

[4] and [5] introduce a modern approach to technical documentation in Diagrams as Code, emphasizing the integration of diagrams directly within codebases. This methodology enhances maintainability by ensuring that diagrams evolve alongside the software, reducing inconsistencies and improving clarity for developers. Unlike traditional static documentation, diagram-as-code tools facilitate automation, version control, and collaboration, aligning with contemporary DevOps and CI/CD practices. This work builds on the broader movement of infrastructure-as-code and declarative documentation, reinforcing the necessity of dynamic, code-driven representations in software engineering workflows.

Unlike these efforts, which treat diagrams as static artifacts or require manual intervention for versioning, our framework automates diagram synchronization with code changes and leverages Neo4j for semantic and structural retrieval. Furthermore, while Structurizr and IcePanel support predefined modeling constraints, our use of Claude and ChatGPT introduces flexibility for handling both forward and reverse transformations guided by prompt engineering.

[6] mentions the emerging technique of diagrams as code, emphasizing how it enhances collaboration and accuracy in architectural modeling by enabling version control, automated updates, and integration with other development tools, thus improving the consistency and efficiency of technical documentation.

[7] introduces a novel approach to software documentation using interactive node-link diagrams to improve codebase exploration and navigation, offering a more dynamic alternative to traditional static documentation and API references.

[8] applies category theory to computability, modeling programs as categorical objects and computations as diagrammatic transformations. The monoidal computer is introduced to unify program evaluation and composition, emphasizing structural universality. The concept of program-closed categories suggests that all well-ordered universal programming languages are fundamentally equivalent in expressive power.

The paper [9] introduces a standardized graphical language tailored to deep learning architectures, leveraging category theory to address the shortcomings of existing diagramming methods by offering precise, compositional structures that closely align with implementation and mathematical analysis.

In related works [10], DisCoPy builds upon the established graphical languages for monoidal categories, integrating and extending them within Python to facilitate applications across quantum computing, linguistics, and other domains by pro-

viding a unified, flexible toolkit for string diagrams and their corresponding algorithms.

[11] emphasizes the role of category theory and string diagrams in advancing XAI by providing a compositional framework that enhances the interpretability of AI models, particularly through the use of visual and formal methods that allow for clearer reasoning about the internal processes and structure of models, thus addressing key challenges in the field of XAI.

[12] extends previous approaches to rendering string diagrams, particularly by integrating algebraic methods from category theory and recursive techniques, highlighting how these innovations improve upon existing graph layout algorithms in terms of computational efficiency and visual clarity.

These systems focus primarily on model presentation and navigation. In contrast, our framework operationalizes diagram generation as a dynamic and composable process integrated with codebases. We go beyond visual clarity to emphasize interactivity, updatability, and semantic search, making our system more suitable for long-lived, evolving projects with collaborative teams.

The use of automated tools, specifically PyTorchKGT, for visualizing computational graphs of deep neural networks is explored in [13], enhancing the efficiency and clarity of model development and analysis.

The PyTorchViz tool [14], which is utilized in [13] for visualizing computational graphs of deep neural networks, generates detailed and accurate representations of PyTorch models, aiding in the analysis and debugging of complex neural architectures.

The thesis [15] draws on existing research into the visualization and communication of neural network systems through diagrams, extending the discussion by empirically evaluating the effectiveness of current diagramming practices and proposing a new framework aimed at improving the clarity and usability of scholarly diagrams in neural network publications.

[16] extends previous research on diagramming techniques for data science by integrating entity-relationship (ER) notation with data flow diagrams, building on prior frameworks that focus on generating diagrams from source code and enhancing them for better collaboration and automation in data pre-processing tasks.

Prior research in digital image processing and deep learning models [17], particularly focusing on the challenges and advancements in the recognition of single-line diagrams (SLDs) using convolutional neural networks (CNNs) and transformers improves the accuracy and efficiency of diagram interpretation.

[18] automates the creation of mathematical diagrams, particularly focusing on how systems like Penrose leverage domain-specific languages (DSLs) to translate mathematical notation into visual representations, thus addressing challenges in generating clear and accurate diagrams from abstract concepts.

Existing approaches to conversational interaction with graph databases as described in [19] often involve pipeline-based

question-answer systems that rely on semantic parsing, query rewriting, and context tracking. The Claude LLM is combined with specialized local servers and plugins (e.g. Model Context Protocol - MCP) that automatically formulates Cypher queries and interprets results. These advances build on earlier efforts in knowledge-based question answering and graph-based analytics, but extend them by enabling more natural, dialog-driven exploration of complex datasets.

Anthropic’s recent work [20] on “building blocks” for generative AI offers a targeted focus on controllability, interpretability, and safety—pillars essential for cultivating trustworthy, high-performing models. This initiative aligns with broader developments across both industry and academia, where researchers are refining advanced model capabilities while striving to ensure robust, transparent governance.

Prior work on AI-assisted development tools broadly investigated automation features like auto-complete, static analysis, and refactoring, underscoring potential gains in efficiency and code quality. More recently, as described in [21], LLMs have ushered in new paradigms of human-AI interaction, including conversational interfaces and context-aware recommendations. Researchers have begun to explore how to optimize these interactions in terms of trust, usability, and developer control—areas critical for navigating the complexities introduced by increasingly intelligent, generative technologies.

III. ARCHITECTURE OVERVIEW

Figure 1 illustrates a distributed workflow system built on top of Anthropic’s Claude LLM that integrates multiple MCP servers, including Neo4j, GitHub, and the local file system, to facilitate seamless interaction between tools, codebases, and documentation. The architecture supports efficient updates and version control through GitHub while leveraging Neo4j for query and data visualization tasks. This setup simplifies collaboration and sharing via HTTPS, while enabling smooth integration with IDEs and tools on the host machine. The system also ensures the synchronization of various file types, including .py, .puml, .cypher and .svg diagrams, improving the readability and maintainability of the codebase. Both PlantUML and TikZ also allow for the inclusion of mathematical formulas in their respective diagrams.

GitHub Repository

The GitHub repository acts as a central storage hub, containing various types of files such as .puml, .tex, .png, .svg, .adoc, .md, .py, .cypher, etc. serving to store and version the project’s artifacts. A dedicated folder named “blueprints” is created within this repository to house all related diagrams. This folder includes sources like TikZ, PlantUML, and Structurizr DSL, and exports these sources as .svg files, which can contain navigable links and be incorporated into a static website. For more intricate diagrams that require detailed control, including orthogonal arrows, TikZ is preferred over PlantUML. TikZ provides greater customization than the automatic layout algorithm available in PlantUML. Additionally, .png files

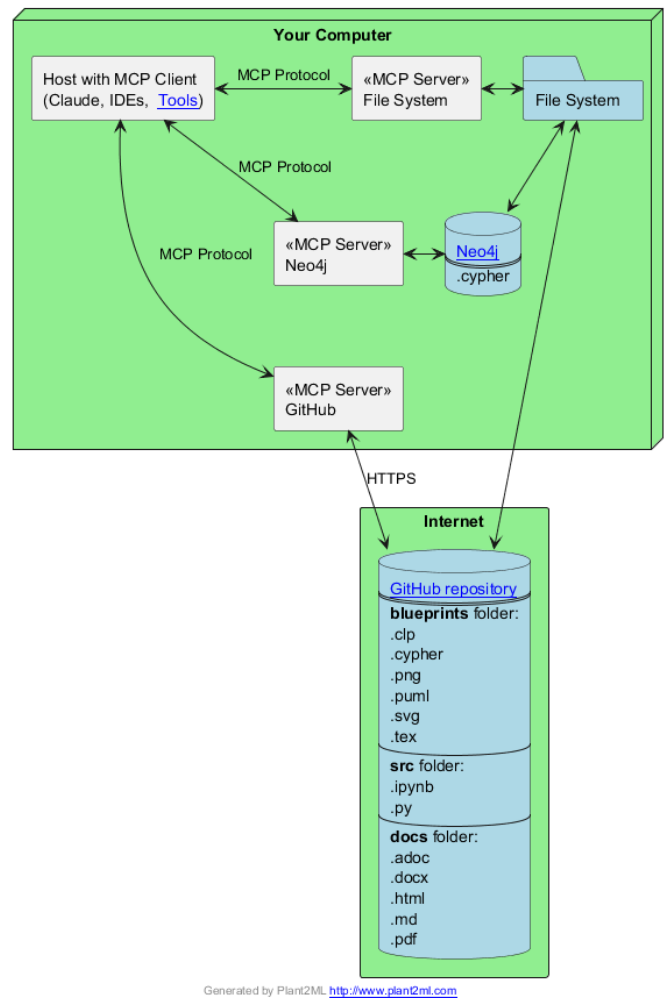


Fig. 1. Architecture

are prepared for inclusion in LaTeX documents or Overleaf projects.

Neo4j

Neo4j is a highly scalable, native graph database designed to model, store, and query data in the form of interconnected nodes and relationships. Its schema-free, property graph model makes it particularly effective for applications requiring dynamic and complex data structures, such as recommendation systems, fraud detection, and knowledge graphs. In addition to its traditional graph capabilities, Neo4j supports similarity and vector searches, enabling advanced analytics and machine learning applications. Through the integration of vector embeddings, Neo4j can store and compare multi-dimensional data points.

Neo4j is used to provide unified storage for both PlantUML and TikZ diagrams. The diagrams, consisting of vertices and edges, map naturally to this graph database. It supports Cypher queries, keyword and similarity searches, as well as natural language queries via the Claude Neo4j MCP server.

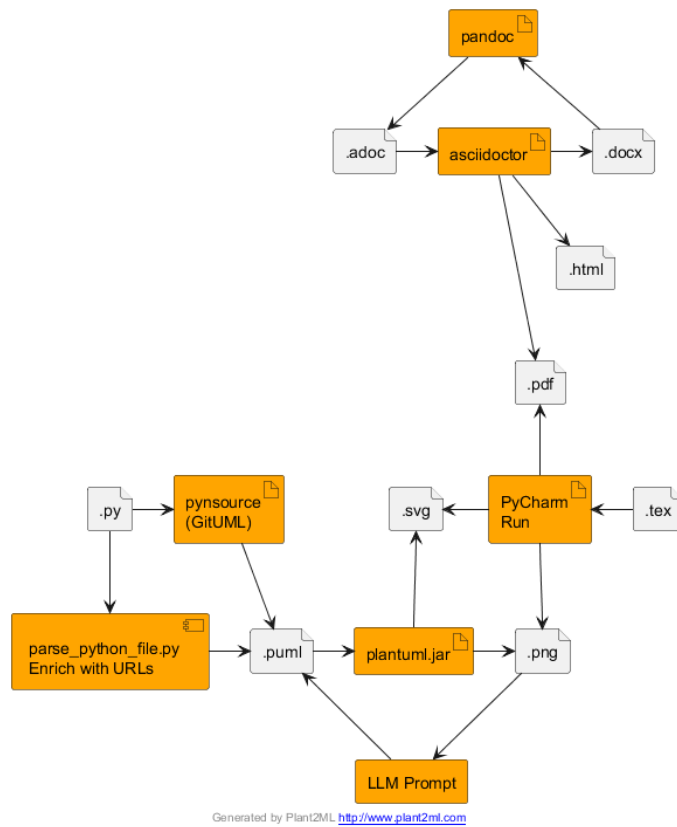


Fig. 2. Tools

Diagrams are stored in Neo4j using a property graph model, where each diagram is represented as linked nodes with metadata attributes such as ‘name’, ‘type’, ‘created_at’, and ‘source_format’. Relationships are defined between diagrams and their components (e.g., ‘CONTAINS’ edges linking to class or method nodes) to enable fine-grained semantic retrieval. To support natural language queries, Claude Desktop converts English prompts into Cypher queries using the MCP protocol. The Claude-MCP server processes user input, maps it to an intent, generates the Cypher query, and then returns structured results for visualization or further processing.

Another advantage of using Neo4j is the ability to graphically explore graphs, including pinning nodes or collapsing and expanding nodes within relationships.

Tools

Figure 2 depicts a documentation and visualization pipeline that transforms Python code and textual inputs into various output formats, enhancing the comprehensibility and usability of software artifacts. The workflow begins with Python files, processed through tools like pynsource (or GitUML) to generate PlantUML .puml files, enriched further with URLs for interactivity. These .puml files are converted to .svg and .png formats using plantuml.jar, facilitating visual representation. Concurrently, text documents in .adoc format are processed by AsciiDoctor and Pandoc to produce .docx, .html, and .pdf

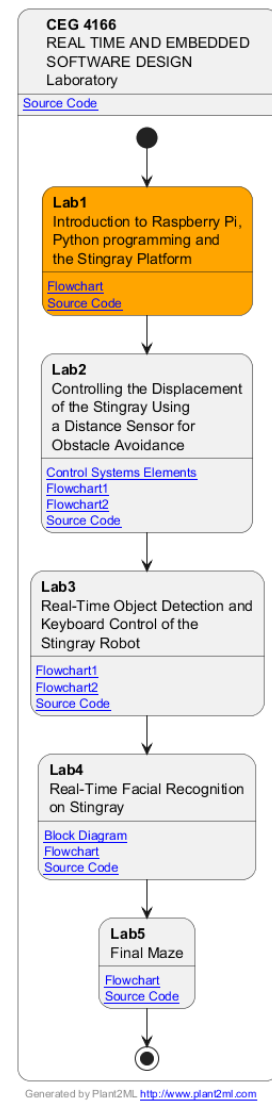


Fig. 3. Laboratory Overview

outputs. The integration with PyCharm IDE’s Run ensures seamless execution of this pipeline, while LLM prompts utilize the generated visuals for further analysis and content generation. This interconnected framework streamlines the generation, visualization, and distribution of documentation across multiple formats.

IV. DIAGRAMS AS CODE

The diagram in Figure 3 exemplifies the application of the "diagrams as code" methodology within the CEG 4166 Real-Time and Embedded Software Design laboratory. It visualizes the sequential structure of lab activities, starting from the introduction to Raspberry Pi and Python programming to the final maze project. Each lab module includes links to source code, flowcharts, and supporting materials, facilitating clear communication of the development and execution stages. By using diagrammatic representations generated from code, the workflow maintains consistency, ensures up-to-date

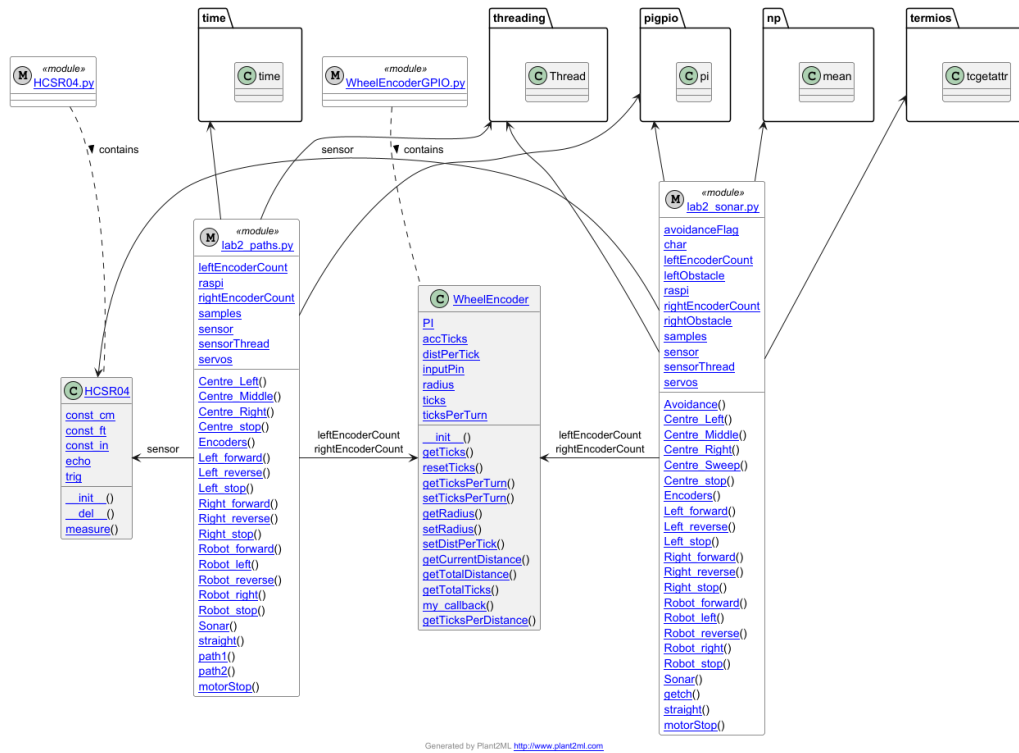


Fig. 4. Lab 2 Source Code

visualizations, and enhances accessibility for collaboration and educational purposes. This approach aligns with modern practices in software documentation by providing dynamic and version-controlled artifacts.

V. CODE AS DIAGRAMS

The diagram in Figure 4 illustrates the modular architecture of a system involving Python modules and their interdependencies. Key modules include `HCSR04.py`, `WheelEncoderGPIO.py`, `lab2_paths.py`, and `lab2_sonar.py`, with each module encapsulating distinct functionalities. The `HCSR04.py` module handles ultrasonic sensor operations, with methods like `measure()` for distance calculations. The `WheelEncoderGPIO.py` module defines the `WheelEncoder` class, managing parameters like ticks and distance per tick, and includes methods for calculating encoder-based distances. The `lab2_paths.py` module serves as the central hub, integrating the ultrasonic sensor and wheel encoder data for path planning and navigation, while the `lab2_sonar.py` module incorporates obstacle avoidance logic. External libraries, such as `threading`, `pigpio`, `numpy`, and `termios`, are used for multi-threading, GPIO control, numerical operations, and terminal settings, respectively. The diagram effectively highlights the flow of data and method interactions, such as `leftEncoderCount` and `rightEncoderCount`, demonstrating how modules collaborate to implement the system's navigation and obstacle detection features.

In terms of navigation, the user starts with the Laboratory Overview statechart in Figure 3 and moves to the "Source Code" diagram in Figure 4 by clicking the corresponding link.

If a link in the "Source Code" PlantUML diagram is clicked, the user is redirected to the corresponding GitHub source code fragment that is highlighted. Another type of navigation is between diagrams. For example: starting with the architecture diagram from Figure 1 by clicking on the "Tools" link, the diagram in Figure 2 is shown.

VI. LLM PROMPTS

Users interact with the Claude and ChatGPT LLMs to generate code, diagrams from code, which are then stored in the "blueprints" folder. ChatGPT was used for generating python code used across the project, while Claude for its integration with the local computer with Neo4j, GitHub and file system via MCP.

A series of prompts are presented, each designed to utilize LLMs for a range of tasks pertaining to the processing and visualization of software architecture and code documentation. The actual prompts are displayed below using the Courier font.

- 1) **List all diagrams in the database** Claude Desktop prompt that gets mapped to Neo4j Cypher and returns the list of diagrams.
List all diagrams in the database
- 2) **Diagrams related to Wheel** Claude Desktop prompt that returns the diagrams containing the string Wheel.
List all diagrams related to Wheel
- 3) **Generate Cypher file** An one shot prompt containing an example of transformations.

```

Given PlantUML:
<paste example PlantUML diagram>
and corresponding Cypher file:
<paste example Cypher file>
Generate the Cypher file
for the given file:
<paste PlantUML diagram>

```

- 4) **Collapse PlantUML diagram nodes:** The final prompt asks ChatGPT to take an expanded PlantUML diagram and simplify it by collapsing all nodes, thus creating a more compact visual representation suitable for overviews or high-level presentations.

```

Given:
<paste expanded PlantUML diagram>
Collapse all nodes.
Output as PlantUML.

```

VII. EMPIRICAL EVALUATION

The proposed method has been evaluated across three domains: (1) time series forecasting using the AutoGluon machine learning framework, (2) the CEG 4166 Real-Time and Embedded Software Design laboratory at the University of Ottawa, and (3) an ongoing Operational Transformation (OT) project focused on collaborative code and diagram editing.

In our prior work [22], we introduced a novel approach that integrates Finite State Machines (FSMs) with PlantUML to automate aspects of machine learning pipelines. In particular, we demonstrated how FSMs can be used to generate visual representations of workflows in AutoGluon, thereby improving the interpretability and transparency of AI models.

Initial attempts to generate complete diagrams directly using ChatGPT yielded limited success, as the output often lacked structural coherence or completeness. Effective diagram generation requires decomposing the intended visual into individual components, each specified through targeted prompts. These components are then assembled manually to produce a coherent and accurate PlantUML or TikZ diagram.

VIII. CONCLUSION

This paper presented a framework that marries LLMs with diagram-as-code principles for enhanced software engineering workflows. By allowing code to be visualized as diagrams—and, conversely, diagrams to be treated as maintainable code—the approach increases both the interpretability and maintainability of complex systems. Storing artifacts such as PlantUML or TikZ in Neo4j enables graph, keyword and similarity searches, fostering an environment of interactive, graph-based collaboration. Integrated version control via GitHub and automated LLM-driven prompts further simplify diagram generation and updates. Together, these features address a critical need for clarity and agility in real-time and embedded software design. Future work includes expanding vector-based indexing and retrieval of both diagrams and source code, enhancing real-time collaboration, and applying the framework to broader software domains. A corpus of .png and PlantUML/TikZ pairs

is being built. It will be used in RAG search to enable the automatic generation of diagram code from its image.

REFERENCES

- [1] S. Brown, “Software architecture for developers,” 2018. [Online]. Available: <https://c4model.com>
- [2] L. Zimmermann. Study buddy - chatbots as lecture companions, using LLMs and retrieval augmented generation. [Online]. Available: <https://eprints.ost.ch/id/eprint/1176/>
- [3] R. Parsons, R. Laycock, M. Fowler, B. Böckeler, and B. Byars, “Thoughtworks technology radar.”
- [4] O. K. Hosney, *Diagrams as Code: Exploring Mermaid, PlantUML, D2 and Generating Diagrams using AI LLMs*. Independently published.
- [5] O. Hosney. Beyond markdown: A new era of diagram-as-code” — book spotlight. [Online]. Available: <https://medium.com/@omkamal/beyond-markdown-a-new-era-of-diagram-as-code-book-spotlight-33d5308cabd6>
- [6] Diagrams as code | technology radar | thoughtworks canada. [Online]. Available: <https://www.thoughtworks.com/en-ca/radar/techniques/diagrams-as-code>
- [7] A. Štěpánek. [2407.21621] interactive diagrams for software documentation. [Online]. Available: <https://arxiv.org/abs/2407.21621>
- [8] D. Pavlovic, *Programs as Diagrams: From Categorical Computability to Computable Categories*, 2023rd ed. Springer Nature.
- [9] V. Abbott, “Neural circuit diagrams: Robust diagrams for the communication, implementation, and analysis of deep learning architectures.” [Online]. Available: <http://arxiv.org/abs/2402.05424>
- [10] A. Touni, R. Yeung, B. Poór, and G. de Felice, “DisCoPy: the hierarchy of graphical languages in python.” [Online]. Available: <http://arxiv.org/abs/2311.10608>
- [11] S. Tull, R. Lorenz, S. Clark, I. Khan, and B. Coecke, “Towards compositional interpretability for XAI.” [Online]. Available: <http://arxiv.org/abs/2406.17583>
- [12] C. Rubio-Madriral and J. Hedges, “Rendering string diagrams recursively.” [Online]. Available: <http://arxiv.org/abs/2404.02679>
- [13] M. Preuhsler, “Applying automatic visualisation to deep neural network development.”
- [14] S. Zagoruyko, “szagoruyko/pytorchviz,” original-date: 2018-01-30T15:37:55Z. [Online]. Available: <https://github.com/szagoruyko/pytorchviz>
- [15] G. Marshall. An investigation into the scholarly usage of neural network systems diagrams. [Online]. Available: <https://research.manchester.ac.uk/en/studentTheses/an-investigation-into-the-scholarly-usage-of-neural-network-syste>
- [16] C. Ordonez, R. Varghese, N. Phan, and W. Macyna, “Growing a FLOWER: Building a diagram unifying flow and ER notation for data science,” in *Proceedings of the 2024 Workshop on Human-In-the-Loop Data Analytics*. ACM, pp. 1–8. [Online]. Available: <https://dl.acm.org/doi/10.1145/3665939.3665958>
- [17] L. Yang, J. Wang, J. Zhang, H. Li, K. Wang, C. Yang, and D. Shi, “Practical single-line diagram recognition based on digital image processing and deep vision models,” vol. 238, p. 122389. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0957417423028919>
- [18] K. Ye, W. Ni, M. Krieger, D. Ma’ayan, J. Wise, J. Aldrich, J. Sunshine, and K. Crane, “Penrose: from mathematical notation to beautiful diagrams,” vol. 39, no. 4. [Online]. Available: <https://dl.acm.org/doi/10.1145/3386569.3392375>
- [19] M. Hunger. Claude conversing with neo4j via MCP. [Online]. Available: <https://medium.com/neo4j/claude-conversing-with-neo4j-via-mcp-5419d24bb7a0>
- [20] C. Greyling. Anthropic’s research on GenAI building blocks. [Online]. Available: <https://cobusgreyling.medium.com/anthropics-research-on-genai-building-blocks-bb20803a2bef>
- [21] C. Treude and M. A. Gerosa, “How developers interact with AI: A taxonomy of human-AI collaboration in software engineering.” [Online]. Available: <http://arxiv.org/abs/2501.08774>
- [22] M. Trifan, B. Ionescu, and D. Ionescu, “A combined finite state machine and PlantUML approach to machine learning applications,” in *2023 IEEE 17th International Symposium on Applied Computational Intelligence and Informatics (SACI)*, pp. 000 631–000 636, ISSN: 2765-818X. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10158543>